User Manual
AetherMMS

Space-Time GNSS Planning for Mobile Mapping Systems

Authors
Dr. Matteo Cappuccio
Prof. Stefano Gandolfi

Institution
DICAM - University of Bologna - Italy

Release
Version 1.0

2026

“Because geomatics engineering is engineering itself,
and every successful survey begins with proper planning.”

AetherMMS v1.0

Contents

1 Introduction

AetherMMS is a software tool designed to support the planning of GNSS-assisted Mobile Mapping System (MMS) surveys in complex environments. The software evaluates expected satellite visibility conditions along a predefined trajectory and provides quantitative indicators to support mission planning before field operations.

Reliable GNSS positioning remains a fundamental component of MMS workflows, particularly in urban areas where buildings, vegetation and transportation infrastructures may reduce satellite visibility. AetherMMS integrates satellite orbit propagation, terrain modelling and urban environment reconstruction to simulate the visibility conditions experienced by a moving platform throughout the survey mission, identifying potentially critical trajectory sections and evaluating the temporal evolution of satellite availability and positioning geometry.

In addition, AetherMMS supports mission planning by comparing alternative acquisition periods and highlighting the time windows expected to provide the most favourable GNSS conditions, assisting surveyors and engineers in selecting suitable survey strategies before data collection.

1.1 Major Features

The main features of AetherMMS v1 are:

1. Space-time GNSS planning for Mobile Mapping System surveys in urban and partially obstructed environments;
2. Simplified 3D reconstruction of urban scenarios from terrain, building and vegetation datasets;
3. GNSS satellite visibility analysis along MMS trajectories through ray-tracing techniques, with LOS, NLOS and vegetation-intersection classification;
4. Computation of GNSS visibility indicators, including LOS satellite count, PDOP and Sky Visibility Index (SVI);
5. Time-dependent mission-planning analysis for the identification of favourable and critical acquisition windows;
6. Multi-constellation support for GPS and Galileo almanac processing;
7. Differential GNSS base-station distance evaluation for RTK and post-processing applications;
8. Interactive visualization of visibility conditions, LOS/NLOS classifications, vegetation-affected signals, PDOP evolution, skyplots, SVI temporal behaviour and critical trajectory sections;
9. Export of GNSS planning results and trajectory statistics for documentation and further analysis.

1.2 Contact Us

For technical information, scientific collaborations or bug reports related to AetherMMS, please contact:

Dr. Matteo Cappuccio
matteo.cappuccio@unibo.it
DICAM – University of Bologna

Prof. Stefano Gandolfi
stefano.gandolfi@unibo.it
DICAM – University of Bologna

2 Fundamentals

AetherMMS simulates GNSS visibility conditions along a planned MMS trajectory. Each point of the route is associated with a survey epoch, while the selected GNSS satellites are propagated from the loaded almanacs. Satellite-receiver geometry is transformed into the local antenna reference frame and evaluated against terrain, buildings, vegetation and GNSS-denied sections.

Unlike a static skyplot analysis, visibility is evaluated continuously along the trajectory, associating each route segment with the expected number of usable satellites, LOS/NLOS conditions, vegetation degradation, PDOP and Sky Visibility Index values.

2.1 Trajectory time parameterization

The MMS trajectory is parameterized as a function of time using the route length and the user-defined vehicle speed. The main mission simulation adopts a fixed 1 Hz step, so receiver position and satellite geometry are updated once per second. The survey-time prediction stage (Section 2.8), which repeats the simulation for many candidate start times, samples the trajectory with a coarser uniform subset of epochs.

$$d(t) = vt \tag{1}$$

$$T = \frac{L}{v} \tag{2}$$

where $d(t)$ Travelled distance at epoch t
v Vehicle speed
t Elapsed survey time
T Estimated survey duration
L Total trajectory length

For each epoch, the receiver position is interpolated along the trajectory according to the travelled distance. Survey date and acquisition time are treated as UTC/GNSS time to maintain temporal consistency throughout the simulation.

2.2 GNSS orbit determination

Satellite positions are computed from the GNSS almanacs loaded by the user. The current version of AetherMMS supports GPS YUMA almanacs and Galileo XML almanacs.

For each selected constellation and simulation epoch, the almanac parameters are propagated to estimate satellite positions in the Earth-Centered Earth-Fixed (ECEF) reference frame.

Satellite motion is described through the Keplerian orbital parameters

$$(a, e, i, \Omega, \omega, M) \tag{3}$$

where a Semi-major axis
e Orbital eccentricity
i Orbital inclination
Ω Longitude of ascending node
ω Argument of perigee
M Mean anomaly

The mean motion is computed as

$$n = \sqrt{\frac{\mu}{a^3}} \tag{4}$$

and the mean anomaly at epoch t becomes

$$M(t) = M_0 + n\, t_k \tag{5}$$

The propagation interval t_k is computed from the GPS time of week as

$$t_k = t - t_0, \qquad t_k \in [-302400,\ +302400] \text{ s} \tag{6}$$

where t_k is reduced by one week (604800 s) whenever it exceeds half a week. The almanac week number is not used in the propagation: the loaded almanac must therefore refer to the same GPS week as the simulated survey date, otherwise the satellite geometry of a different day is silently reproduced. The same applies to Galileo almanacs.

Kepler's equation is solved iteratively to obtain the eccentric anomaly:

$$M = E - e \sin E \tag{7}$$

A Newton iterative scheme is used:

$$E_{k+1} = E_k - \frac{E_k - e \sin E_k - M}{1 - e \cos E_k} \tag{8}$$

The true anomaly is then computed as

$$\nu = \arctan 2 \left(\sqrt{1 - e^2} \sin E,\ \cos E - e \right) \tag{9}$$

The orbital radius is

$$r = a(1 - e \cos E) \tag{10}$$

2.3 From orbital plane to ECEF

After the anomaly and orbital radius have been computed, the satellite position is first expressed in the orbital plane.

The argument of latitude is

$$u = \nu + \omega \tag{11}$$

and the orbital-plane coordinates become

$$x_{orb} = r \cos u \tag{12}$$

$$y_{orb} = r \sin u \tag{13}$$

The longitude of the ascending node is corrected to account for Earth rotation during orbital propagation:

$$\Omega(t) = \Omega_0 + (\dot{\Omega} - \omega_E)t_k - \omega_E t_0 \tag{14}$$

where $\Omega(t)$ Corrected longitude of ascending node
Ω_0 Longitude of ascending node at reference epoch
$\dot{\Omega}$ Right ascension rate
ω_E Earth rotation rate
t_k Elapsed time from almanac reference epoch
t_0 Almanac reference time

The orbital coordinates are then rotated into the ECEF reference frame:

$$X_s = x_{orb} \cos \Omega - y_{orb} \cos i \sin \Omega \tag{15}$$

$$Y_s = x_{orb} \sin \Omega + y_{orb} \cos i \cos \Omega \tag{16}$$

$$Z_s = y_{orb} \sin i \tag{17}$$

The propagated satellite position in the ECEF frame is therefore

$$\mathbf{r}_s = \begin{bmatrix} X_s & Y_s & Z_s \end{bmatrix}^T \tag{18}$$

2.4 Receiver position and local ENU frame

For each trajectory epoch, the receiver longitude and latitude are interpolated along the trajectory (Section 2.1), while the user-defined antenna height provides the height of the antenna phase centre.

[1]

Receiver coordinates are converted from geodetic coordinates to ECEF using the WGS84 ellipsoid:

$$X_r = (N + h)\cos\varphi\cos\lambda \tag{19}$$
$$Y_r = (N + h)\cos\varphi\sin\lambda \tag{20}$$
$$Z_r = \left[N(1 - e_W^2) + h\right]\sin\varphi \tag{21}$$

$$N = \frac{a_W}{\sqrt{1 - e_W^2\sin^2\varphi}} \tag{22}$$

where λ Receiver longitude
φ Receiver latitude
h Receiver antenna height
a_W WGS84 semi-major axis
e_W WGS84 eccentricity
N Prime vertical radius of curvature

The receiver-satellite vector in ECEF is

$$\Delta\mathbf{r}_{ECEF} = \mathbf{r}_s - \mathbf{r}_r = \begin{bmatrix}\Delta X & \Delta Y & \Delta Z\end{bmatrix}^T \tag{23}$$

This vector is transformed into the local East-North-Up (ENU) frame:

$$E = -\sin\lambda\Delta X + \cos\lambda\Delta Y \tag{24}$$
$$N = -\sin\varphi\cos\lambda\Delta X - \sin\varphi\sin\lambda\Delta Y + \cos\varphi\Delta Z \tag{25}$$
$$U = \cos\varphi\cos\lambda\Delta X + \cos\varphi\sin\lambda\Delta Y + \sin\varphi\Delta Z \tag{26}$$

The origin of the ENU frame is the antenna phase centre at the current trajectory epoch: as the platform moves, the frame is re-instantiated at every epoch at the interpolated receiver position. The E and N axes span the local horizon plane (geodetic East and North), while the U axis points along the outward ellipsoidal normal at the receiver location. The resulting ENU vector defines the local satellite direction with respect to the MMS antenna: its azimuth and elevation (Section 2.5) are reused as the ray direction within the relative-elevation urban scene of Section 2.6.

[1] Strictly, the height h in the geodetic-to-ECEF conversion is the *ellipsoidal* height, which differs from the orthometric (DTM) height by the geoid undulation. AetherMMS uses the antenna height above the local terrain: satellite look angles are insensitive to the receiver height at the satellite distance (a tens-of-metres error shifts the elevation by less than an arcsecond), and the obstruction test operates on heights relative to the mean DTM elevation, so the undulation cancels. The method is thus independent of the vertical datum and requires no geoid model, applying unchanged to any survey area.

2.5 Azimuth, elevation and visibility mask

From the ENU components, AetherMMS computes satellite azimuth, elevation and range:

$$Az = \arctan 2(E, N) \tag{27}$$

$$El = \arctan 2\left(U, \sqrt{E^2 + N^2}\right) \tag{28}$$

$$\rho = \sqrt{E^2 + N^2 + U^2} \tag{29}$$

Satellites below the user-defined elevation mask are discarded before the obstruction analysis. The remaining satellites are then tested against the reconstructed urban environment.

2.6 Ray-tracing visibility classification

For each present satellite, AetherMMS generates a local antenna-to-satellite ray using the computed azimuth and elevation angles.

Because GNSS satellites are located at very large distances from the receiver, the local ENU direction is sufficient for the obstruction analysis.

The ray height at a horizontal distance s from the antenna is

$$z_{ray}(s) = h_{ant} + \tan(El)s \tag{30}$$

where $z_{ray}(s)$ Ray height at distance s
h_{ant} Antenna height
El Satellite elevation angle
s Horizontal distance from the antenna

All vertical quantities share the same local reference: elevations sampled from the bare-earth Digital Terrain Model (DTM) are expressed relative to the mean DTM elevation of the trajectory, and each building or tree is placed on this relative terrain through a per-feature base elevation z_{base}, directly comparable with h_{ant}. Each ray is tested within a maximum horizontal distance of 1.2 km (parameter `max_ray_km` in the toolbox).

2.6.1 Terrain and building obstruction

Terrain obstruction is evaluated by sampling the DTM profile along the ray azimuth at the DTM ground resolution (5 m for the example dataset): if the ray height falls below the terrain elevation plus a 0.5 m clearance, the satellite is classified as NLOS. Tying the sampling step to the DTM resolution makes the analysis adapt automatically to any input DTM, from high-resolution LiDAR models to coarser regional or global grids.

Building obstruction is evaluated through the intersection between the horizontal ray projection and the building footprints. If s^* is the distance of an intersection with a footprint, the satellite is NLOS when the ray is below the roof at that point:

$$z_{ray}(s^*) \leq z_{base} + h_{build} \tag{31}$$

where h_{build} is the building height above its local base elevation z_{base}.

2.6.2 Vegetation model

Each tree is modelled as two coaxial vertical cylinders sharing the centre of the crown: a crown cylinder of radius R_c between heights $z_{base} + h_{cb}$ and $z_{base} + h$, and a trunk cylinder of radius R_t between z_{base} and $z_{base} + h_{cb}$, where h is the tree height and h_{cb} the crown base height.

Given the tree centre at horizontal distance d_c and bearing β_c from the antenna, the centre is decomposed into along-ray and cross-ray components:

$$s_{\parallel} = d_c \cos(\beta_c - Az) \tag{32}$$
$$s_{\perp} = d_c \, |\sin(\beta_c - Az)| \tag{33}$$

A cylinder of radius R is crossed when $s_{\perp} \leq R$, between the entry and exit distances

$$s_{1,2} = s_{\parallel} \mp \sqrt{R^2 - s_{\perp}^2} \tag{34}$$

The satellite is classified as vegetation-degraded when the ray height interval $[z_{ray}(s_1),\ z_{ray}(s_2)]$ overlaps the vertical extent of either cylinder. Terrain and building obstructions take precedence: a satellite is labelled vegetation-degraded only when no hard obstruction is found along the ray.

2.6.3 Obstacle geometry from input attributes

Obstacle dimensions are derived from the input attributes. Building heights are read from common height fields or from eaves/ground elevation pairs (e.g. `quota_gronda–quota_piede`); a 16 m default is applied when no readable height is available. Tree heights accept numeric or class-based values (e.g. "Cl4: 16mt–23mt", parsed as the class minimum). When no crown diameter is provided, it is estimated from the trunk diameter d_t (cm) as $d_{crown} = \min(\max(d_t/7,\ 2.5),\ 10)$ m; the crown base height is $h_{cb} = \max(1.8,\ 0.38\,h)$ and the trunk radius is bounded to 0.08–0.45 m.

2.6.4 GNSS-denied sections

Tunnel and covered-road candidates are retrieved from OpenStreetMap within the analysis buffer, filtered semantically (tunnels, covered carriageways, overlapping structures) and retained only

when they actually cross the trajectory. Epochs falling inside the resulting geometries are flagged as GNSS-denied: satellite geometry may still be propagated, but those epochs are classified as unavailable for GNSS positioning.

2.7 PDOP and Sky Visibility Index

At each epoch, AetherMMS computes the number of present, LOS, NLOS and vegetation-degraded satellites.

PDOP is computed only from LOS satellites. At least four LOS satellites are required; otherwise, the geometric solution is considered unavailable.

For each LOS satellite, the geometry matrix is built from the local direction cosines:

$$\mathbf{g}_j = \begin{bmatrix} \cos El_j \sin Az_j & \cos El_j \cos Az_j & \sin El_j & 1 \end{bmatrix} \tag{35}$$

The covariance matrix is obtained as

$$\mathbf{Q} = (\mathbf{G}^T\mathbf{G})^{-1} \tag{36}$$

and the Position Dilution of Precision is

$$PDOP = \sqrt{Q_{xx} + Q_{yy} + Q_{zz}} \tag{37}$$

Lower PDOP values indicate stronger satellite geometry. In AetherMMS an epoch is considered usable when at least four LOS satellites are available and the LOS-based PDOP does not exceed 8; these thresholds drive both the outage statistics and the continuity term of the planning score (Section 2.8).

The Sky Visibility Index is computed as

$$SVI = \frac{N_{LOS}}{N_{present}} \tag{38}$$

where N_{LOS} is the number of unobstructed satellites and $N_{present}$ is the total number of propagated satellites above the elevation mask.

In GNSS-denied sections, the Sky Visibility Index is set to zero.

These indicators are associated with trajectory epochs and used to generate satellite-count charts, PDOP evolution plots, skyplots and trajectory quality maps.

2.8 Survey time prediction

One of the main functions of AetherMMS is the estimation of suitable acquisition windows: since satellite geometry continuously changes during the day, the same trajectory may experience

significantly different GNSS visibility conditions depending on the selected start time. The prediction workflow therefore simulates the complete survey trajectory for multiple candidate start times distributed throughout the day, evaluating LOS availability, PDOP and Sky Visibility Index epoch by epoch.

2.8.1 Planning score definition

The simulated epochs are first separated into GNSS-denied epochs (tunnel and covered-road sections) and N_{valid} valid epochs. Since GNSS-denied sections are fixed properties of the route, independent of the start time, they are excluded from the ranking score and reported separately as diagnostic indicators (tunnel and overall outage percentages). All score terms are computed over the valid epochs only.

The global planning score is defined as

$$Score = 100\,(0.40\,S_{LOS} + 0.25\,S_{PDOP} + 0.35\,C) \tag{39}$$

Since the three terms are heterogeneous quantities (a satellite count, an inverse-quality factor and a percentage), each is first mapped onto the unit interval, so that the weights retain their intuitive meaning.

The availability term normalizes the mean number of LOS satellites $\overline{N}_{LOS}$, saturating at ten satellites:

$$S_{LOS} = \min\left(\frac{\overline{N}_{LOS}}{10},\ 1\right) \tag{40}$$

The score grows linearly up to ten LOS satellites, beyond which additional redundancy brings negligible planning benefit and the term saturates at 1; the usability floor of four satellites is enforced separately by the continuity term. For example, an average of 8 LOS satellites yields $S_{LOS} = 0.8$, while 12 yields $S_{LOS} = 1$.

The geometry term maps the mean LOS-based PDOP, $\overline{PDOP}$, onto the unit interval between the usability threshold $PDOP_{max} = 8$ and the optimal value 2:

$$S_{PDOP} = \min\left(\max\left(\frac{8 - \overline{PDOP}}{6},\ 0\right),\ 1\right) \tag{41}$$

If no valid PDOP can be computed along the window, $S_{PDOP} = 0$.

The continuity term C is the fraction of valid epochs offering usable GNSS conditions. A valid epoch is classified as critical when fewer than four LOS satellites are available, when the LOS-based PDOP cannot be computed, or when it exceeds the fixed threshold $PDOP_{max} = 8$:

$$C = 1 - \frac{N_{crit}}{N_{valid}} \tag{42}$$

where N_{crit} is the number of critical valid epochs. High continuity values indicate temporally stable mission conditions with limited GNSS outages along the trajectory.

The Sky Visibility Index is provided as an auxiliary urban-visibility indicator but is not directly included in the temporal ranking score in order to avoid redundancy with LOS-based availability estimation.

2.8.2 Two-stage window search

The prediction workflow is organized into two stages.

First, AetherMMS performs a coarse daily preview between 06:00 and 22:00 using a 30-minute temporal spacing. To limit computational cost, this stage adopts a reduced number of route samples and omits the vegetation test, which is the most expensive obstruction check.

The most favourable and most critical preview windows are then used as refinement seeds: additional start times are tested at ±15 minutes around each seed using a denser route sampling and the complete obstruction model, including vegetation. The final best and worst windows are selected from the refined candidates. The number of seeds, the refinement offset and the sampling densities are configurable in the Python toolbox.

For each candidate acquisition window, the software reports:

- planning score;
- average LOS satellite count;
- average PDOP;
- average SVI;
- outage percentage;
- GNSS-denied (tunnel) percentage.

The best and worst windows correspond to the highest and lowest scores among the refined candidates and should be interpreted as relative planning indicators rather than direct estimates of positioning accuracy.

3 Program Structure and Workflow

AetherMMS follows a sequential workflow for GNSS-aware Mobile Mapping System planning: data ingestion, urban-scene reconstruction, satellite propagation, ray-tracing visibility analysis and mission-quality evaluation (Fig. 1).

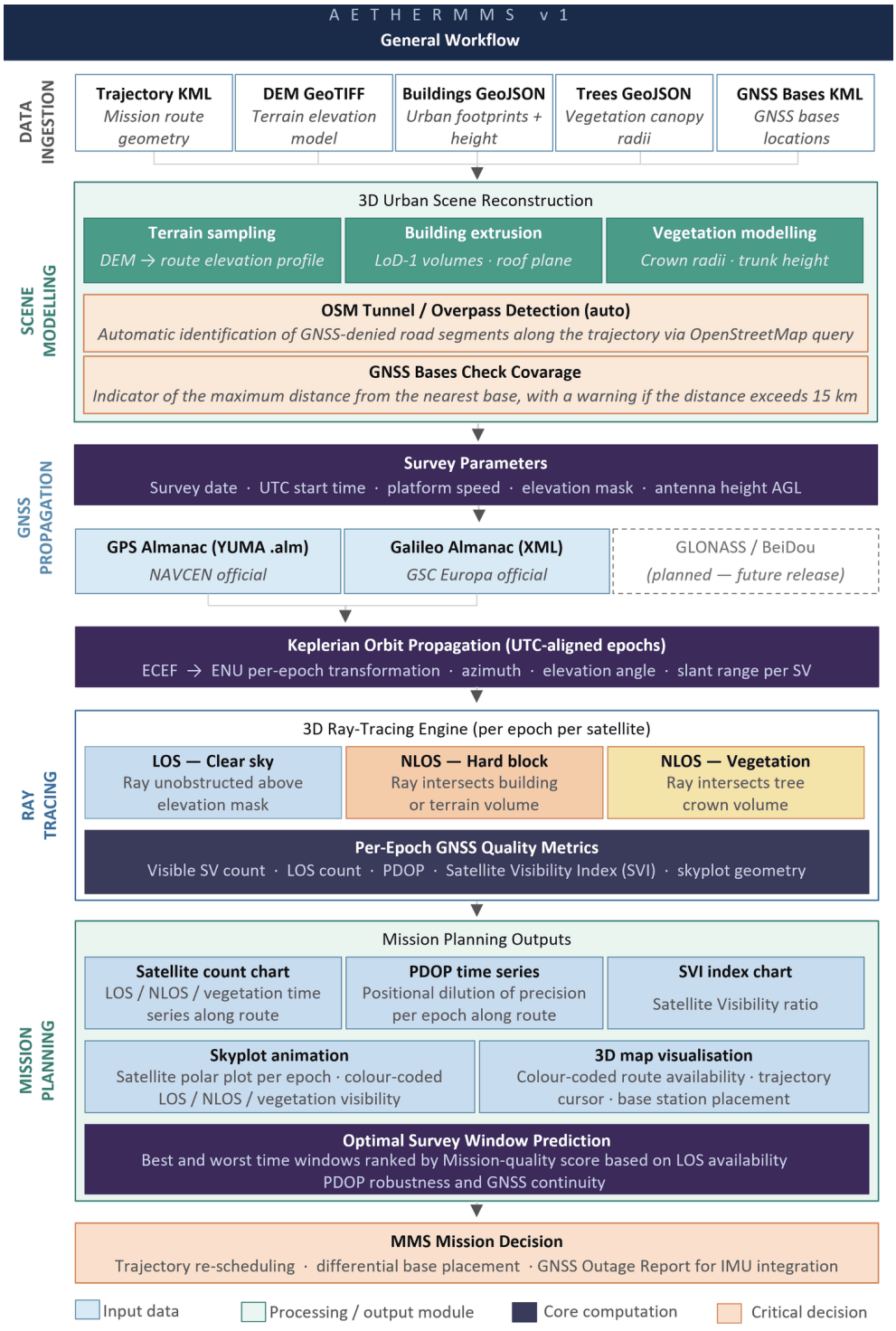

Figure 1: General workflow of AetherMMS v1.

3.1 Data ingestion

The first stage loads the survey trajectory, the Digital Terrain Model (DTM), building and vegetation layers and the GNSS almanacs. The datasets are checked and organized to reconstruct the survey environment; their consistency and spatial coverage directly affect the reliability of the subsequent visibility analysis.

3.2 Urban scene modelling

AetherMMS reconstructs a simplified three-dimensional representation of the survey environment: terrain elevations are derived from the DTM, building footprints are converted into volumetric obstacles and vegetation layers are represented through simplified canopy models. Tunnel and covered-road sections are automatically retrieved from OpenStreetMap and treated as potentially GNSS-denied areas (Sections 2.6 and 4.4). Although intentionally lightweight, the reconstruction provides a realistic geometric framework for the ray-tracing analysis.

3.3 GNSS propagation

Satellite positions are computed for each simulated epoch from the loaded almanacs, following the orbital propagation and the ECEF/ENU transformations introduced in Sections 2.2–2.5. The current implementation supports GPS and Galileo constellations.

3.4 Ray-tracing engine

For each propagated satellite above the selected elevation mask, a receiver-to-satellite ray is generated from the local ENU geometry and intersected with the reconstructed environment according to the visibility model of Section 2.6. Satellites are classified as:

- **LOS**, when the satellite is fully visible;
- **NLOS**, when the signal is blocked by terrain or buildings;
- **vegetation-degraded**, when the ray intersects vegetation elements.

This classification identifies the potentially critical trajectory sections before field operations.

3.5 Best temporal window prediction

Since satellite geometry changes continuously throughout the day, the same MMS trajectory may experience significantly different GNSS conditions depending on the survey start time. AetherMMS simulates the complete survey for multiple candidate acquisition windows and ranks them with the planning score of Section 2.8, identifying the most favourable and most critical periods.

3.6 Mission planning outputs

The final stage generates the mission-planning outputs: satellite-count time series, skyplots, trajectory quality maps, graphical visibility indicators and planning summaries, computed according to the formulations of Section 2.7. Their combined interpretation supports the selection of more reliable acquisition windows and reduces the risk of prolonged GNSS degradation during the survey mission.

4 Technical Requirements

AetherMMS is available in two complementary implementations, based on the same methodological framework and requiring the same categories of input data: a standalone Python toolbox for scientific processing, distributed through the GPS Solutions Toolbox collection, and an interactive web application for visualization and exploratory assessment of GNSS visibility conditions.

4.1 Python Toolbox Requirements

The toolbox has been tested on Microsoft Windows 10/11, Ubuntu Linux and macOS. It requires Python 3.10 or later together with the dependencies listed in the accompanying `requirements.txt` file:

```
pip install -r requirements.txt
```

No proprietary applications or external processing engines are required.

4.2 Web Application Requirements

The web implementation requires a modern browser supporting JavaScript and WebGL; Google Chrome and Microsoft Edge are recommended. Rendering performance may vary with the graphics hardware when visualizing large urban environments.

4.3 Input Data Requirements

Both implementations require user-provided geospatial and GNSS datasets: GNSS almanacs, Digital Terrain Models (DTM), building footprints, vegetation layers, MMS trajectories and optional GNSS reference station locations. The quality and reliability of the visibility analysis depend directly on the accuracy, completeness and consistency of these datasets.

The elevation raster must be a bare-earth DTM: buildings and vegetation are modelled separately as vector obstacles, so a Digital Surface Model (DSM) would account for them twice and falsify the obstruction analysis.

4.4 Network Requirements

Most processing stages run entirely offline. An internet connection is required only for the automatic tunnel and covered-road detection, which queries the OpenStreetMap Overpass servers (the Python toolbox can reuse a previously generated local cache through the `osm_use_cache` and `osm_cache_file` configuration keys), and for the optional online almanac retrieval in the web application. With a tunnel cache and locally stored almanac files, complete analyses can be performed without network access.

4.5 Computational Requirements

All computations are performed internally by AetherMMS, without external GIS applications, databases or cloud services. Computational time increases with trajectory length, satellite availability, urban complexity and the number of simulated epochs.

The recommended minimum configuration is a multi-core 64-bit processor, 16 GB RAM, 2 GB of available disk space and hardware-accelerated graphics for the web application. A workstation-class system is recommended for extensive urban environments, long trajectories or large mission-planning campaigns.

5 Using the Python Toolbox

The Python implementation provides the complete computational environment for GNSS visibility analysis and mission planning: a set of modular scripts coordinated through a single execution file and a plain-text configuration file.

5.1 Project Structure

The toolbox is distributed with source files, configuration files, a sample dataset and an output directory.

A typical installation contains the following structure:

```
AetherMMS/
|-- source/
|-- config/
|-- Data/
|-- results/
```

The `Data` directory contains a demonstration dataset provided for testing purposes. Users may replace it with their own project files, updating the corresponding paths in the configuration file.

5.2 Software Modules

The computational workflow is implemented through a set of dedicated Python modules.

`runAetherMMS.py` Main execution script responsible for coordinating the complete processing workflow.

`buildingcity.py` Imports geospatial datasets, reconstructs the urban scene and prepares the objects required for visibility analysis.

`satellitepropagation.py` Parses GNSS almanacs and computes satellite positions throughout the simulation period.

`bestwindowsprediction.py` Performs visibility assessment, obstruction classification and mission-planning analyses.

`plotting.py` Generates figures, maps, plots and summary reports.

5.3 Configuration File

All processing parameters are defined within:

`config/config.txt`

The configuration file follows a simple key–value structure and can be edited with any text editor: each line has the form `key = value`, the text to the right of a `#` character is treated as an inline comment, and blank lines or `[section]` headers are purely organizational and ignored by the parser. When a value is left empty, the documented default is applied.

Input file names are resolved inside the `data_dir` folder (absolute paths are also accepted), while the output directory is resolved against the project root: the configuration therefore remains fully portable when the project folder is moved or copied. All quantities use fixed units — metres for distances and heights, degrees for angles, km/h for the vehicle speed — and dates and times are interpreted as UTC (`YYYY-MM-DD`, `HH:MM:SS`). Boolean keys accept `true/false`.

The `survey` and `ray_tracing` sections directly affect the scientific results; the `best_worst_windows` section controls the temporal prediction only, and `figures` influences the graphical outputs alone. The planning sampling densities (`planning_preview_max_epochs`, `planning_refinement_max_epochs`) and the preview spacing trade accuracy for runtime: larger values sample the trajectory and the day more finely at proportionally higher computational cost, while the default values reproduce the behaviour of the web application.

Table 1 summarizes the available parameters. An alternative configuration file can be selected at run time through the `-config` flag described in Section 5.5.

Table 1: Configuration parameters of `config/config.txt`.

Section	Key	Meaning
folders	`data_dir`, `output_dir`	input and output folders
inputs	`dem_geotiff`	DTM GeoTIFF
	`trajectory_kml`	MMS trajectory (KML)
	`buildings_geojson`, `trees_geojson`	building and tree datasets (GeoJSON)
	`bases_kml`	optional GNSS bases (KML)
	`gps_yuma`, `galileo_xml`	GPS YUMA and Galileo XML almanacs
survey	`lidar_range_m`	analysis buffer half-width around the route
	`antenna_height_m`	antenna height above local route elevation
	`date_utc`, `start_time_utc`	survey epoch (UTC)
	`speed_kmh`	vehicle speed
	`elevation_mask_deg`	satellite elevation mask
	`step_sec`	main mission sampling step (1 = 1 Hz)
	`constellations`	enabled constellations (GPS, Galileo)
osm	`osm_enabled`	enable tunnel/covered-road detection
	`osm_use_cache`, `osm_cache_file`	reuse a local OSM cache (offline runs)
	`overpass_timeout_s`, `overpass_endpoints`	Overpass query settings
best_worst_windows	`planning_enabled`	run the temporal prediction
	`planning_start_time_utc`, `planning_end_time_utc`	daily scan interval (default 06:00–22:00)
	`planning_step_minutes`	preview spacing (default 30 min)
	`planning_seed_count`	best/worst seeds refined (default 6)
	`planning_refine_offset_minutes`	refinement offsets around each seed (default ±15 min)
	`planning_preview_max_epochs`, `planning_refinement_max_epochs`	route samples per preview/refinement pass (default 42/90)
	`planning_include_vegetation`	vegetation test in the refinement pass
ray_tracing	`max_ray_km`	obstruction search distance (default 1.2 km)
figures	`figure_dpi`	PNG resolution
	`selected_skyplot_mode`, `selected_skyplot_value`	skyplot epoch selection (time or distance)

5.4 Input Datasets

The analysis requires a DTM (GeoTIFF), an MMS trajectory (KML), building and vegetation datasets (GeoJSON), GPS YUMA and Galileo XML almanacs and optional GNSS base station locations (KML).

GNSS almanacs should refer to the same GPS week as the simulated survey date (Section 2.2); when comparing toolbox and web-application results, the same almanac files should be used in both environments.

5.5 Running AetherMMS

After preparing the datasets and updating the configuration file, the software can be executed from a command-line terminal.

From the project directory:

```
python source/runAetherMMS.py
```

The runner accepts two optional command-line flags:

`-config <path>` Path to an alternative configuration file. The default is `config/config.txt`; relative paths are resolved against the project folder, so the software can be executed from any working directory.

`-no-figures` Skips the generation of PNG figures and of the HTML report, producing only the CSV tables and the metadata file. Useful for batch processing or when the graphical outputs are not required.

For example:

```
python source/runAetherMMS.py --config config/myproject.txt --no-figures
```

The execution automatically performs:

1. urban scene preparation;
2. GNSS almanac parsing;
3. satellite orbit propagation;
4. visibility and obstruction analysis;
5. mission-planning evaluation;
6. report and figure generation.

5.6 Generated Outputs

All outputs are automatically stored in the directory specified in the configuration file. The CSV tables (`results/csv`) are:

- `epochs.csv`: per-epoch metrics (visible/LOS/NLOS/vegetation counts, PDOP, SVI, GNSS-denied flag);
- `satellite_visibility.csv`: per-satellite looks (azimuth, elevation, visibility status, blocking cause);
- `pdop_timeseries.csv`: LOS-based PDOP evolution (all constellations, GPS-only, Galileo-only);
- `best_window_summary.csv`: ranked acquisition windows with all score components;
- `best_window_trajectory.csv`, `worst_window_trajectory.csv`: per-epoch metrics of the two selected windows;
- `metadata.txt`: run configuration and scene statistics.

Unless `-no-figures` is used, the run also produces seven diagnostic figures in `results/png` (satellite counts, PDOP profile, SVI, skyplot, trajectory quality, temporal window scores, best/worst comparison) and the offline HTML report `AetherMMS_report.html`.

5.7 Typical Workflow

A standard analysis consists of preparing the datasets, updating `config/config.txt`, executing `runAetherMMS.py` and reviewing the generated outputs, refining the configuration when additional analyses are required.

6 Graphical User Interface

The AetherMMS web interface provides an interactive environment for GNSS-aware Mobile Mapping System mission planning, organized into two components: the **3D Visualization Environment**, dedicated to the inspection of the survey scenario, and the **Mission Control Panel**, which contains the tools for data loading, survey configuration, GNSS analysis and result inspection. Fig. 2 shows the general layout of the interface.

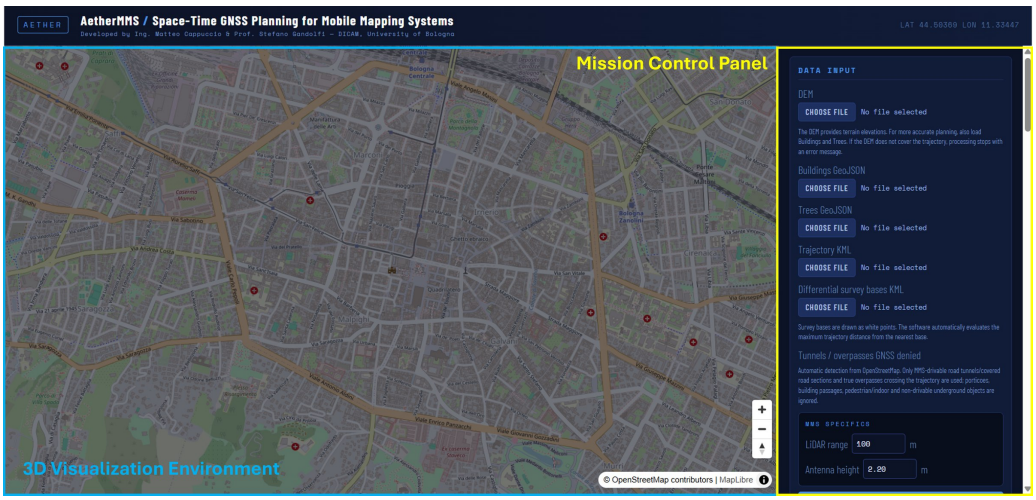

Figure 2: General layout of the AetherMMS graphical interface showing the 3D Visualization Environment and the Mission Control Panel.

6.1 3D Visualization Environment

The 3D Visualization Environment provides an interactive view of the survey corridor: MMS trajectory, terrain morphology, buildings, vegetation, GNSS survey bases, tunnel sections and visibility-analysis results can be inspected before and after processing (Fig. 3).

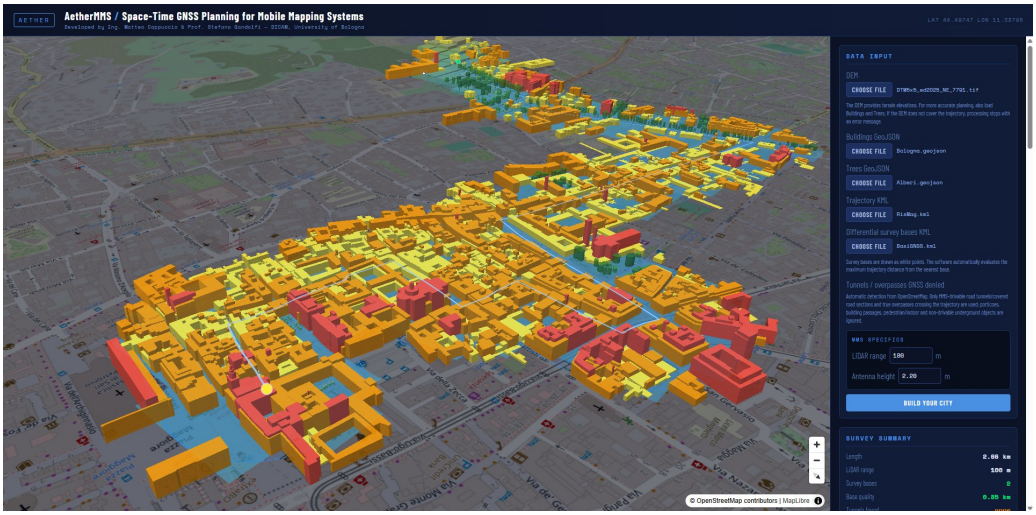

Figure 3: Detailed visualization of buildings, vegetation and trajectory buffer.

6.2 Mission Control Panel

The Mission Control Panel follows the main AetherMMS workflow, from input data loading to mission evaluation (Figs. 4–8).

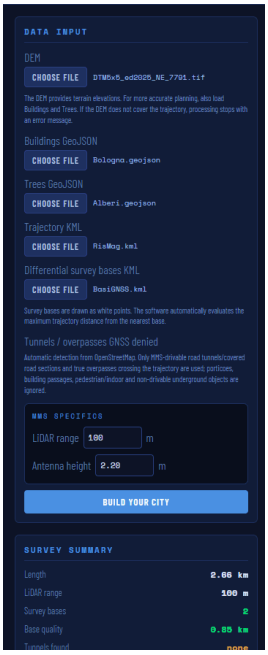

Figure 4: Data ingestion and MMS configuration section.

Figure 5: Survey specifications and almanac management section.

The **Data Input** section loads DTMs, trajectories, buildings, vegetation datasets and GNSS survey bases, together with MMS-specific parameters such as LiDAR range and antenna height; the `Build your city` command then generates the urban model used for the visibility analysis.

The **Survey Specifications** section defines the survey date, start time, vehicle speed, elevation mask and active GNSS constellations (GPS and Galileo). Almanacs can be imported from local files or retrieved online; the almanac week should match the survey date (Section 2.2) and, for consistency with the Python toolbox, the same files should be loaded in both environments.

The **GNSS Processing** section contains the operational commands: `Calculate 3D Intersections` starts the visibility analysis, while `Predict Best Survey Time` compares alternative acquisition windows. After processing, the software reports visibility statistics, LOS/NLOS and vegetation classifications and mission-planning indicators, together with LOS satellite counts, PDOP evolution, skyplots and SVI time series. The analysed trajectory is colour-coded by LOS availability: red for 4 or fewer LOS satellites or GNSS-denied segments, orange for 5–7, and green for more than 7 LOS satellites.

Finally, the lower section of the panel (Fig. 8) contains the visualization controls, the symbology legend and a log console reporting the main operations performed by the software.

Figure 6: GNSS processing controls and planning results.

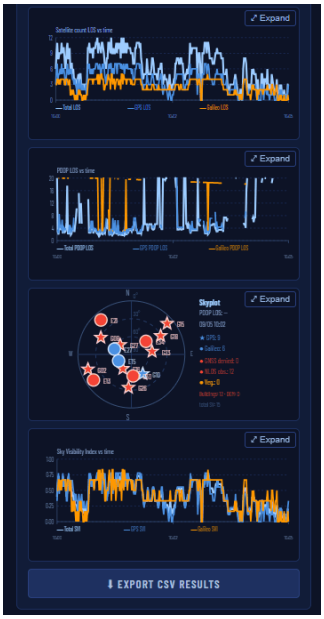

Figure 7: GNSS charts and graphical analysis outputs.

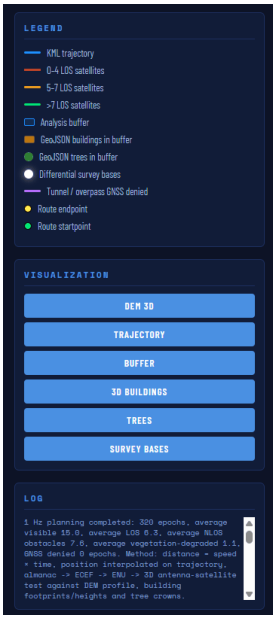

Figure 8: Legend, visualization controls and processing log.

7 Limitations and Future Developments

AetherMMS is intended as a mission-planning and visibility-analysis tool: the generated indicators should be interpreted as relative measures of expected survey conditions rather than direct estimates of positioning accuracy.

7.1 Current Limitations

The urban environment is represented through simplified geometric models: buildings are treated as fully blocking obstacles and vegetation is described through simplified canopy geometries. The visibility analysis is based on geometric line-of-sight assessment and does not explicitly model multipath, signal diffraction, atmospheric effects, receiver tracking limitations or INS error propagation.

Satellite positions are computed from GNSS almanacs, fully adequate for planning purposes but less accurate than precise orbit products. The propagation uses the time of week only and does not verify the almanac week number, so the temporal consistency between almanac and survey date is left to the user (Section 2.2).

The current version supports GPS and Galileo constellations only, and processing time may increase for large urban environments or long survey trajectories.

7.2 Future Developments

Future developments will focus on full multi-constellation support (GLONASS and BeiDou), advanced vegetation attenuation models, simplified multipath estimation, urban-canyon quality indicators and visibility reliability assessment. Later versions may also incorporate simplified INS error modelling, support for precise orbit products, dedicated tools for LiDAR and photogrammetric mission planning, and AI-assisted mission scoring and survey-quality assessment.

References

[1] Kaplan, E. D., and Hegarty, C. J., *Understanding GPS: Principles and Applications*, Artech House, 2nd edition, 2005.

[2] Vallado, D. A., *Fundamentals of Astrodynamics and Applications*, Microcosm Press, 4th edition, 2013.

[3] Hofmann-Wellenhof, B., Lichtenegger, H., and Wasle, E., *GNSS – Global Navigation Satellite Systems: GPS, GLONASS, Galileo, and more*, Springer, 2008.